1. 栈（先进后出）：顺序存储，top, base 指针

2. 队列（先进先出）：

顺序存储：空：top == rear　　满：(rear+1) % size == top

元素个数：(rear - front + size) % size

3. 数组：

行优先：M行N列：a[i][j] 地址 = LOC + (i×N+j) × elem size

列优先：M行N列：b[i][j] 地址 = LOC + (j×M+i) × elem size

4. 树：

(1) ① 结点度：结点孩子个数　　树的度：结点最大度数

分支结点：度>0（非终端）　　兄弟：相同双亲

② 深度：top → bottom　　高度：bottom → top

③ 有序树：左右子树有次序，不可互换

(2) 性质：

① 结点数 = 结点度数和 + 1

② 度为 m 的树中第 i 层至多有 m^{i-1} 个结点

③ 高度为 h 的 m 叉树至多有 $\frac{m^h-1}{m-1}$ 个结点

$(1+m+m^2+\cdots+m^{h-1})$

④ 具有 n 个结点的 m 叉树最小高度为 $\lceil \log_m(n(m-1)+1) \rceil$

$\frac{m^h-1}{m-1}=n$

5. 二叉树：

(1) 满二叉树：结点个数：2^h-1　每层：2^{h-1}

(2) 完全二叉树：最多有1个度为1的结点

当 $i \le \lfloor \frac{n}{2} \rfloor$ 时，为分支节点，否则为叶子结点

(3) 性质：

①叶子结点数 = 度为2结点数 +1，即 $n_0 = n_2 + 1$

$n = n_0 + n_1 + n_2$　　$n = 0 \times n_0 + 1 \times n_1 + 2 \times n_2 + 1$

② 非空二叉树第k层最多有 2^{k-1} 个结点

③ 高度为h的二叉树最多有 2^h-1 个结点

④ 高度为h的完全二叉树，结点n：$2^{h-1} \le n \le 2^h - 1$

具有n个结点的完全二叉树，高度 h：$\lceil \log_2(n+1) \rceil$ / $\lfloor \log_2 n \rfloor + 1$

⑤ 链式存储中：n个结点，非空指针域有n-1个，空指针域有n+1个

6. 树和森林：

(1) 树存储结构：双亲表示法，孩子表示法，孩子兄弟表示法

(2) 树转化为二叉树：

① 兄弟结点之间加一条线

② 对每个结点，只保留它与第一个孩子的连线，抹去与其他孩子的连线

③ 以树根为轴心，顺时针旋转 45°

树 → 二叉树，右子树总是为空

(3) 森林转化为二叉树:

① 将森林中每棵树转换成相应二叉树

② 每棵树的根视为兄弟结点，在每棵树间连线

③ 以第一棵树的根为轴心顺时针旋转45°

除了第一棵树，其余所有都在二叉树根节点的右子树

(4) 二叉树转化为森林:

将(3)中方法由③→①反行其道

(5) 树的遍历:

① 先根遍历: 先根后子树 → 先序 (左)

② 后根遍历: 先子树后根 → 中序 (右)

(6) 森林的遍历:

① 先根遍历: 先根后子树

② 中根遍历: 先子树后根

7. 哈夫曼树:

(1) 结点带权路径长度: 从树的根到任意结点的路径长度与该结点上权值的乘积。

(2) 树的带权路径长度(WPL): 树中所有叶结点的带权路径长度之和

(3) 哈夫曼树: 带权路径最小的二叉树

8. 图论:

(1) 完全图边数: 无向: $\frac{n(n-1)}{2}$ 有向: $n(n-1)$

(2) 度：① 无向图 全部顶点度之和 等于边数 2倍

② 有向图 入度之和 = 出度之和 = 边数

(3) 生成树 边数：n-1

9. 图存储结构：邻接矩阵、邻接表、十字链表、邻接多重表

(1) 邻接矩阵：

① 无向图为对称矩阵

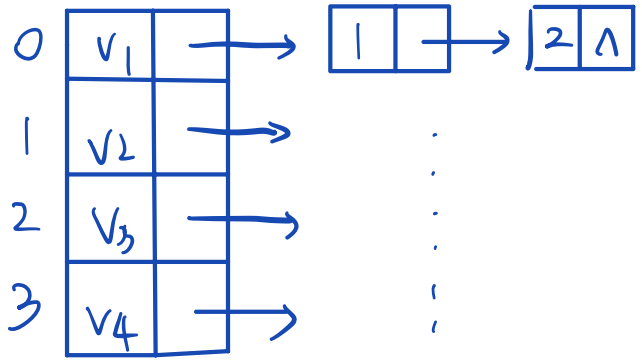
② 与顶点有关，边数无关

③ 带权存权，不带权存 0、1

④ 带权有向图，顶点 i 入度为第 i 列非 0 非 ∞ 元素之和

出度为第 i 行非 0 非 ∞ 元素之和

(2) 邻接表：



所需空间：无向 $O(V+2E)$

有向 $O(V+E)$

① 稀疏图用邻接表节约空间

② 邻接表不唯一

10. DFS → 先序遍历 → 栈　　BFS → 层次遍历 → 队列

判断有环：DFS、Topological sort

11. 查找：

(1) 平均查找长度(ASL)：所有查找过程中进行关键字比较次数的平均值。 $ASL = \sum P_i C_i$

(2) 顺序查找：(从后向前找)

查找成功 $ASL = \sum \frac{1}{n}(n-i+1) = \frac{n+1}{2}$

查找失败 $ASL = n+1$

△链表只能顺序查找

(3) 折半查找(二分查找)：仅适用有序顺序表

查找成功 $ASL = \frac{1}{n}(1\times1 + 2\times2 + 4\times3 + \cdots + h\times2^{h-1})$

$= \frac{n+1}{n}\log_2(n+1) - 1 \approx \log_2(n+1) - 1$

时间复杂度 $O(\log_2 n)$

12. Hash：

collision：(1)封闭哈希法：$H_i = (H(key) + d_i)\%\ m$

① 线性探测法：$d_i = 0, 1, 2, \cdots, m-1$

② 二次探测法：$d_i = 0^2, 1^2, -1^2, 2^2, -2^2 \cdots, k^2, -k^2$

$k \le \frac{m}{2}$

③ 随机探测法

		…
关键字		
比较次数		

查找成功 $ASL = \frac{\text{次数}}{\text{个数}}$

查找不成功：

$ASL = \frac{\text{不成功查找的次数}}{\text{Hash后地址个数}}$

(1)开放哈希法：

①链地址法（链表）

②公共溢出区

查找长度取决于：哈希函数，解决冲突方法，装填因子 $(\alpha = \frac{n}{m})$